

Spaceman Projects

Five-App Ecosystem Master Plan

The authoritative development blueprint for Admin Web, Merchant Web, Customer Web, Customer App and Driver App - including Firebase, Google Cloud / Maps, Paystack, security, data, deployment and start-to-end delivery phases.

ADMIN WEB

MERCHANT WEB

CUSTOMER

DRIVER APP

BACKEND

Architecture principles

- 1

One canonical platform

All channels operate on one Firebase project and one shared domain model.
- 2

Orders are the central transaction

Every app observes or advances the same validated order lifecycle.
- 3

External services enrich through backends

Maps and Paystack never let clients create platform truth directly.
- 4

Security is role and scope based

Auth, App Check, rules, Functions and immutable audit records work together.
- 5

Development is incremental

Current deployed behavior and IDs are preserved through bounded, testable phases.

DOCUMENT SCOPE

- 1

Role experience

Screens, commands, states and UX
- 2

Workflow ownership

Who reads, writes, approves and resolves
- 3

Data contracts

Canonical records, snapshots and enums
- 4

Security boundaries

Rules, Functions, App Check and audit
- 5

Monorepo flow

Shared packages without forced UI reuse
- 6

Development plan

Phases, tests, deployment and Codex prompts

VERSION 1.0 - JULY 2026

Approved Platform Decisions and Non-Negotiables

These decisions govern every future Codex task, migration and deployment.

1 Platform truth

1 One Firebase project

No second backend, duplicate collection set or disconnected test data in production paths.

2 Canonical Firestore records

Orders, users, stores, items and operational records are shared across all five apps.

3 Server timestamps

Lifecycle, SLA, freshness and audit calculations use authoritative time.

4 Historical snapshots

Orders preserve customer/store/item/pricing meaning.

2 Experience boundaries

1 Web stays web-native

Admin, Merchant and Customer Web use appropriate React web UI.

2 Mobile stays native

Customer and Driver apps remain Expo React Native.

3 Share logic, not forced UI

Types, schemas, services and pure utilities belong in shared packages.

4 Preserve existing IDs

Firebase project and EAS project configurations remain intact.

3 Operational decisions

1 Mabopane V1

One delivery area, standard delivery and no surge pricing.

2 Manual workflows remain

Google/API imports are optional accelerators, not replacements.

3 Archive/redact deletion

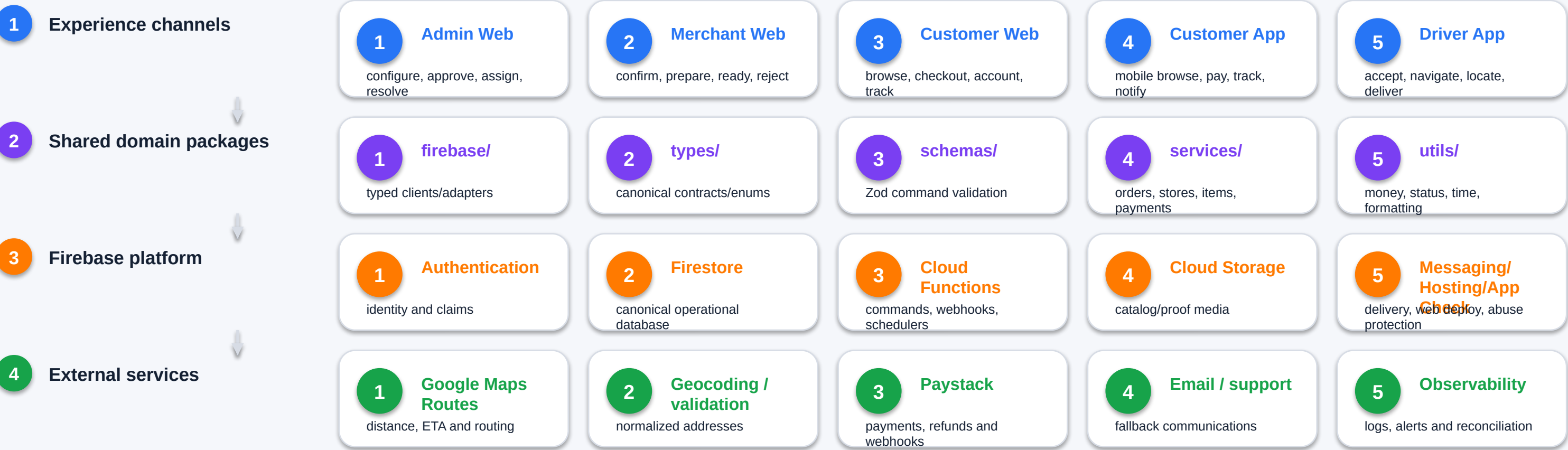
Disable Auth and preserve required order/payment/audit history.

4 Admin control plane

Approvals, roles, refunds, assignment and exceptions remain audited.

Five Apps, Shared Domain and External Platforms

Each app owns a role-specific experience but all data and command boundaries converge on the same backend.



Source-of-truth rule

Clients display and request actions. Firestore contains canonical operational state. Cloud Functions and verified provider events protect privileged mutations and financial truth.

Role, Scope and Capability Matrix

Role guards in the UI are mirrored by Firestore rules and privileged backend checks.

1 Core capability matrix

Capability	Customer	Merchant	Driver	Admin
Browse marketplace	Active catalog	Own preview	Pickup context	All
Place / pay	Own checkout	No	No	Support only
Fulfillment status	Cancel/request	Confirm/prepare/ready	Pickup/on-way/delivered	Override/resolve
Store / items	No	Own scoped	No	Create/import/approve
Assignment	No	Read only	Acknowledge	Assign/reassign
Payment / refund	Pay/request	Read ledger	No	Initiate/resolve
Users / roles	Own profile	Own profile	Own profile	Scoped / super-admin

2 Scope invariants

- 1

Customer ownership
Reads own profile, orders, addresses, favorites and notifications only.
- 2

Merchant store scope
Reads/writes only approved assigned storeIds and allowed fields/transitions.
- 3

Driver assignment scope
Reads assigned work and writes narrow delivery commands/location.
- 4

Admin access
Explicit, audited and separated from Super Admin role assignment.
- 5

System actors
Payment, scheduled jobs and notifications use service identities / Functions.

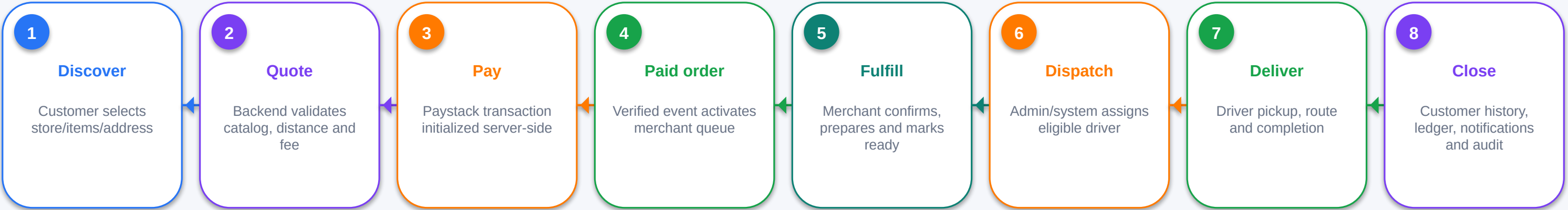
Authorization rule

Hidden buttons are not security. Every protected request is evaluated against authentication, role/status, resource ownership, immutable fields and allowed state transition.

Complete Customer-to-Delivery Sequence

The platform coordinates payment, merchant fulfillment, dispatch and final delivery through one canonical order.

1 Primary sequence



2 Cross-system ownership

Step	Primary writer	Canonical records / events	Other channel reaction
Discover / cart	Customer channel	catalog reads, provisional cart	No order truth yet
Quote	Cloud Function / pricing service	quote, fee-rule version, distance	Customer confirms final total
Payment	Paystack + webhook handler	payment reference/event/status	Merchant remains blocked until paid
Fulfillment	Merchant command Functions	orderStatus + timestamps/history	Customer/admin/driver update
Assignment	Admin/dispatch Function	driverId, assignment version	Driver notification
Tracking	Driver + Maps service	location projection, route/ETA, status	Customer/admin visibility
Completion	Driver command Function	deliveredAt, history, stop tracking	Ledger/analytics/notification

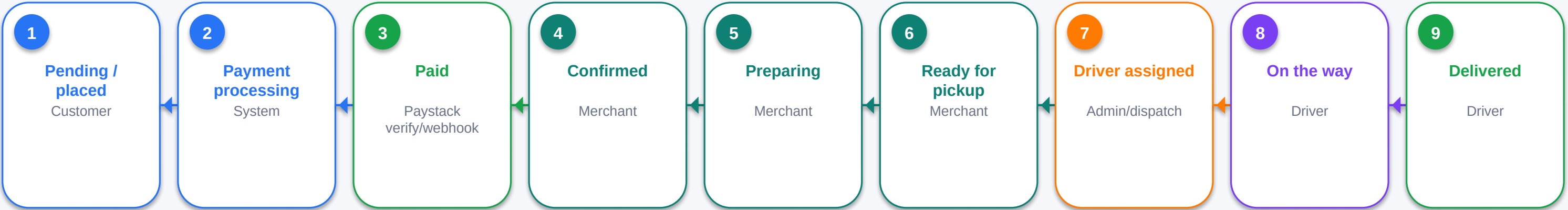
Recovery principle

Every step can be retried or resumed by stable IDs. A client restart, duplicate webhook or repeated command must resolve to the same canonical outcome, not duplicate value.

Canonical Lifecycle, Allowed Writers and Exceptions

All five apps see the same lifecycle; each transition has one permitted actor and server validation.

1 Happy-path lifecycle



2 Exception and needs-action model

Reason / branch	Trigger	Owner / resolution	Required effect
PAYMENT_FAILED	Provider unsuccessful	Customer retry / system	No fulfillment
NO_DRIVER_ASSIGNED	Paid/active order lacks driver	Admin/dispatch	Assign eligible driver
MERCHANT_NOT_RESPONDING	Confirmation SLA exceeded	Admin contacts/resolves	Activity + safe customer status
ORDER_DELAYED	ETA/SLA breached	Admin/driver/merchant	Contact/reassign/update
ADDRESS_PROBLEM	Validation or delivery issue	Customer/admin/driver	Correct or cancel by policy
DRIVER_LOCATION_STALE	Freshness threshold exceeded	Admin/driver	Refresh/contact/reassign
REFUND_PENDING	Refund command not final	Admin/payment handler	Reconcile provider result
CANCELLED / REFUNDED	Approved terminal branch	Backend/provider	Close fulfillment and notify

Transition invariant

No UI may jump Pending to Delivered, mark Paid without verified provider data, overwrite assignment ownership or refund more than the captured amount.

orders/{orderId} Record and Historical Integrity

The order document is both a transaction snapshot and a live fulfillment state.

1 Canonical order sections

1 Identity / channel

orderId, customerId/snapshot, storeId/snapshot, merchantId, driverId/snapshot and source channel.

2 Items

Immutable item snapshots: ID, name, quantity, unit price, modifiers and totals.

3 Pricing

Subtotal, delivery/service fees, discounts, total, currency and rule versions.

4 Payment

Provider, reference, status, verified amount, event and refund references.

5 Delivery

Address snapshot, geo, instructions, distance, route ETA and tracking policy.

6 Lifecycle

Current/previous status, reasons, needsAction, timestamps, history refs and schemaVersion.

2 Data invariants

1 Money is authoritative

Backend calculates and verifies exact amount/currency.

2 Snapshots are immutable

Mutable user/store/item documents never rewrite history.

3 Enums are shared

All apps compile against one status/reason contract.

4 Timestamps are server-owned

SLA and freshness never depend on local clocks.

5 Schema versioning

Readers tolerate approved old versions; migrations are explicit.

6 Audit is append-only

Commands record actor, intent, result and before/after.

Exact path caveat

orders/{orderId} is fixed by the approved blueprint. Other collection names below are a recommended logical contract and must be confirmed against the current repository/deployment before implementation.

Logical Firestore Collections and Ownership

Collections are normalized around ownership and operational queries while orders keep transaction snapshots.

1 Core collection contract

Collection	Primary content	Owner / writer
users/{uid}	role, status, profile, storeScope	self allowed fields + admin/server
stores/{storeId}	identity, geo, hours, source, state	admin + scoped merchant
items/{itemId}	store catalog, media, availability	admin + scoped merchant
orders/{orderId}	canonical transaction/lifecycle	channel transitions + server
feeRules/{ruleId}	versioned store/zone pricing	admin/server
notifications/{id}	recipient event/read state	system + recipient read state
activities/{id}	requests, alerts, issues, status	system/admin + scoped submitter
auditLogs/{id}	immutable command history	server only

2 Supporting records

- 1

importBatches/{id}

Preview, validation, selected rows, results and idempotency.
- 2

paymentEvents/{id}

Provider event ID, signature result, reference and processing status.
- 3

quotes/{id}

Short-lived authoritative checkout quote and expiry.
- 4

driverLocations

Current/order-scoped location projection and freshness metadata.
- 5

assignments / outbox

Versioned assignment and reliable notification/event delivery.
- 6

settlement ledger

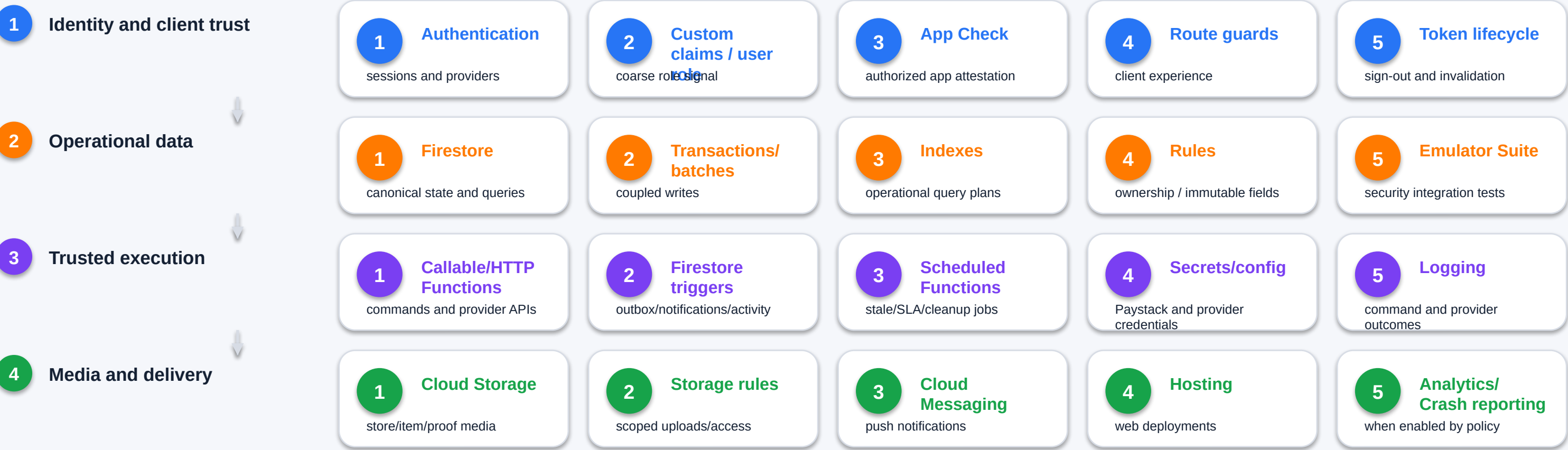
Order financial snapshot, commission and payout/reconciliation state.

Schema discipline

Every mutable document carries schemaVersion, createdAt and updatedAt. Queries use designed indexes and bounded scopes; collection names are not changed casually after deployment.

Firebase Component Responsibilities

Firebase is one integrated platform, but each component has a separate security and operational responsibility.



One-project constraint

Admin, Merchant, Customer Web/App and Driver App must use the same intended Firebase project configuration per environment. Environment separation is explicit, not accidental duplicate production backends.

Authentication, Claims, App Check and Session Rules

Identity, authorization and app attestation solve different problems and must not be confused.

1 Authentication

1 Sign-in identity

Firebase Auth proves the current user/session.

2 Role bootstrap

users/{uid}.role/status plus controlled custom claim.

3 Status enforcement

Suspended/archived users fail operational routes and rules.

4 Token refresh

Role/status changes account for existing sessions.

2 Authorization

1 Rules ownership

Customer uid, merchant storeIds and driver assignment.

2 Protected fields

Payment, claims, totals, assignment and audit are immutable to clients.

3 Function checks

Every command revalidates actor and resource state.

4 Super Admin

Role creation/critical settings restricted and audited.

3 App Check / abuse

1 Attestation

Helps reject traffic from unauthorized app clients.

2 Not user auth

A valid app token does not grant resource ownership.

3 Rate / replay controls

Command IDs, provider event IDs and server limits.

4 Audit alerts

Repeated denials, invalid signatures and anomalies create signals.

Cloud Functions, Commands, Triggers and Scheduled Work

Functions form the privileged command boundary between role-specific clients and protected state.

1

Command families

- 1

Identity commands

Create/assign roles, suspend/unsuspend and archive/redact.
- 2

Store/catalog commands

Import staging, publish, privileged delete and cleanup.
- 3

Order commands

Quote, confirm, prepare, ready, assign, pickup, delivered, cancel override.
- 4

Payment commands

Initialize, verify, webhook, refund and reconciliation.
- 5

Support commands

Create/resolve activities, notifications and audit entries.

2

Execution patterns

- 1

Idempotency

Stable command/event key persisted before external or value-changing mutation.
- 2

Transactions / outbox

Couple canonical change, history, audit and notification intent.
- 3

Retries

Exponential backoff for transient providers; safe repeated commands.
- 4

Scheduled monitors

Merchant SLA, stale driver location, stuck payment/refund and cleanup.
- 5

Failure visibility

Final failures create actionable activities and structured logs.

Function response contract

Return the refreshed canonical resource or a typed error. Clients must not manufacture success from an optimistic local transition.

Cloud Storage, Messaging and Event Delivery

Media and notifications are secondary projections with explicit ownership and cleanup.

1 Cloud Storage

1 Store media

Card/hero images with attribution and responsive derivatives.

2 Item media

Compressed source, thumbnail and stable URL metadata.

3 Proof media

Private, order-scoped and retention-limited when enabled.

4 Upload policy

Type, size, path ownership and malware/content considerations.

5 Cleanup

Delete orphaned assets only after reference checks and audit.

2 Notification architecture

1

Domain event

Order/role/
support change

2

Outbox

Stable event ID
and recipients

3

Dispatch

FCM, web push
or email

4

Delivery
result

sent / failed /
retry

5

In-app record

read state and
deep link

- Use recipient-specific notification documents and never expose another user's messages.
- Deduplicate channels using one logical event ID.
- Invalid device tokens are removed and repeated failures become observable.
- Notifications are not the source of truth; opening the app always refreshes canonical data.

Operational priority

New paid orders, assignments, readiness and critical exception notifications require stronger monitoring than marketing messages.

Address, Route, Distance, ETA and Location Architecture

Google Maps data is normalized through shared services so all apps use one interpretation.

1 Google components

1 Place / address search

User or admin selects a result rather than storing unverified free text alone.

2 Geocoding / reverse geocoding

Resolve validated coordinates and human-readable address context.

3 Address validation

Confirm required delivery components and serviceability.

4 Routes / Route Matrix

Store-to-customer and driver routes, distance and ETA.

5 Maps UI / external navigation

Channel-specific map rendering and driver navigation handoff.

2 Normalized data and governance

Output	Canonical use	Freshness / rule
formattedAddress + place reference	store/customer address snapshot	Validated at selection/checkout
GeoPoint	Serviceability, route and map	Immutable within order snapshot
distanceKm	Delivery fee and operational context	Quote-time source timestamp
routeEtaMinutes	Customer/admin estimate	Refresh under controlled policy
route reference/polyline	Map display/navigation	Do not duplicate blindly in every app
driver location	Tracking projection	Throttled, assignment-scoped, expires
provider status/error	Activity/diagnostics	Never fabricate fallback truth

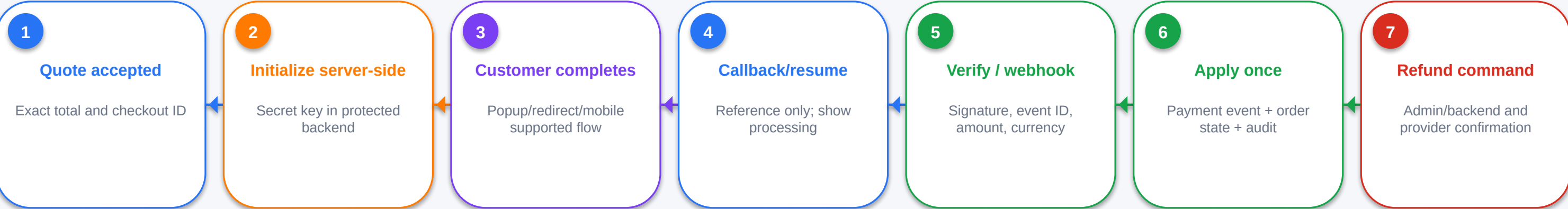
Current API direction

Use current Google Routes / Route Matrix capabilities through a backend adapter rather than allowing each app to implement its own distance/ETA formula or depend on a legacy assumption.

Paystack Initialization, Verification, Webhooks and Refunds

Paystack is the external financial authority; the Spaceman backend maps verified events into canonical payment state.

1 Payment lifecycle



2 Payment protection matrix

Control	Required behavior	Risk prevented
Secret key	Functions/server secrets only	Credential theft and direct provider abuse
Unique reference	Stable checkout/order mapping	Duplicate value and ambiguous reconciliation
Verify amount/currency	Compare provider response to stored quote	Underpayment / tampering
Webhook signature	Validate provider authenticity	Forged paid/refund events
Event idempotency	Persist processed event ID/reference	Replay / duplicate mutation
Callback rule	Never mark paid from client callback alone	False success
Refund validation	Captured state, amount limit and reason	Over-refund / unauthorized refund
Reconciliation job	Resolve stuck processing/pending events	Lost asynchronous updates

Merchant and driver boundary

Neither Merchant Web nor Driver App writes payment state. Merchant fulfillment starts only after the backend exposes a verified paid order.

Delivery Fee, Commission and Settlement Snapshot

Pricing rules are versioned and snapshotted into orders so future configuration changes do not alter history.

1 Delivery fee V1

1 Formula

$\text{clamp}(\text{min}, \text{max}, \text{base} + \max(0, \text{distance} - \text{included}) \times \text{perKm} + \text{small-order surcharge}).$

2 Launch values

Mabopane example: base, included km, per-km rate, threshold/surcharge and min/max.

3 Distance source

Google route distance normalized by backend.

4 Rule version

feeRuleId/version and inputs captured in quote/order.

5 Expansion

Zones, multiple delivery methods and surge only in later approved phases.

2 Order financial snapshot

Component	Purpose	Authority
itemsSubtotal	Customer items before platform fees	Authoritative item snapshots
deliveryFee	Logistics fee by route/rule	Backend pricing service
serviceFee	Optional platform fee	Versioned policy
discounts	Promotion application	Backend eligibility
total	Customer charge	Backend + Paystack amount verify
commissionSnapshot	Store commission basis	Versioned platform/store policy
netMerchantAmount	Ledger calculation	Backend finance projection
refundAmount/status	Returned value	Paystack confirmed event

Finance honesty

Do not display a merchant payout as paid without a trusted settlement reference. V1 may use a read-only order ledger and manual settlement workflow.

Read/Write Ownership by App

Each client has a narrow responsibility over the shared records.

1

Application writes

Actor	Allowed intent	Prohibited direct truth
Customer App/Web	profile fields, checkout request, customer notes, cancel/support request	paid/refund status, final quote, other users
Merchant Web	confirm, preparing, ready, cannot-fulfill, scoped catalog/store edits	payment, assignment, delivered
Driver App	availability, acknowledgement, pickup, location, delivered, issue report	payment, store/catalog, other driver assignment
Admin Web	approvals, assignment, overrides, refund commands, settings	provider event fabrication
Paystack handler	verified payment/refund events	non-payment domain edits
Maps service	validated address/route enrichment	order lifecycle ownership

2

Application reads

- 1

Customer

Public active catalog; own profile, order history, tracking and notifications.
- 2

Merchant

Assigned stores, own catalog, store order queue and ledger projection.
- 3

Driver

Assigned/eligible work, scoped pickup/drop-off data and own notifications.
- 4

Admin

All operationally scoped records, activities, settings and audits.
- 5

System

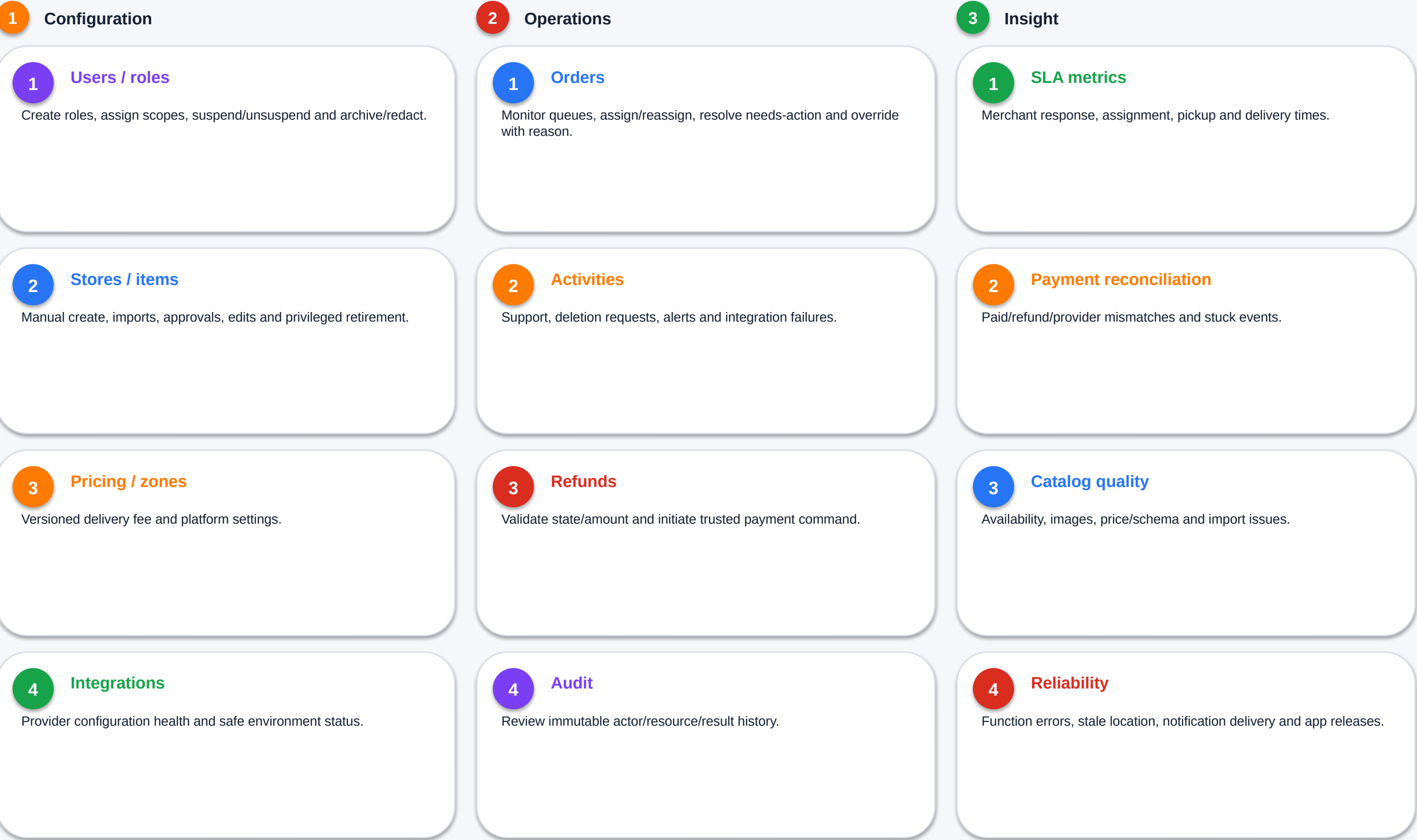
Service reads required for commands, monitoring, reconciliation and cleanup.

Command contract

High-impact writes are expressed as typed commands rather than open-ended document updates. Functions validate intent and return canonical results.

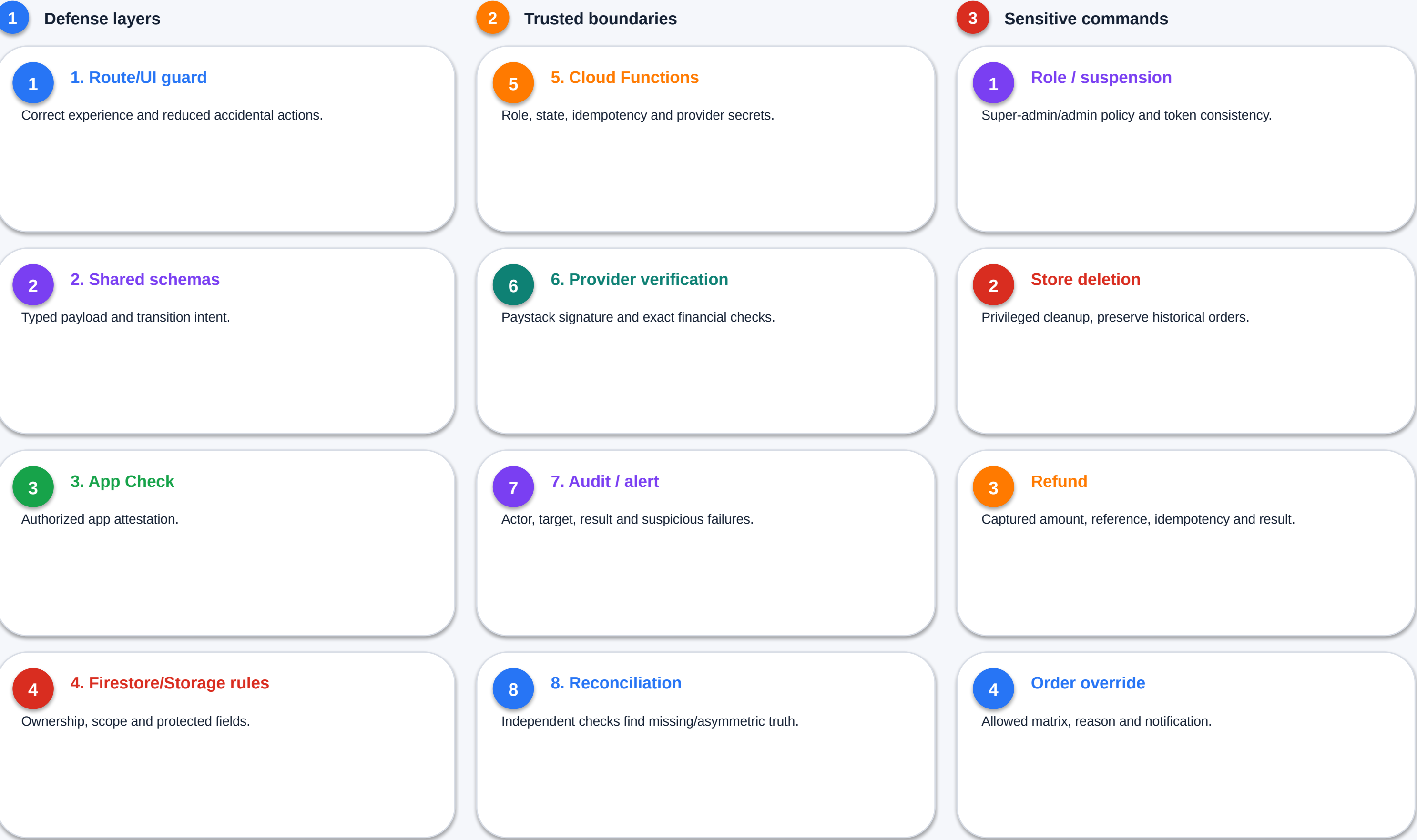
Admin Responsibilities Across the Ecosystem

Admin Web is the operational control plane, not a duplicate of role-specific apps.



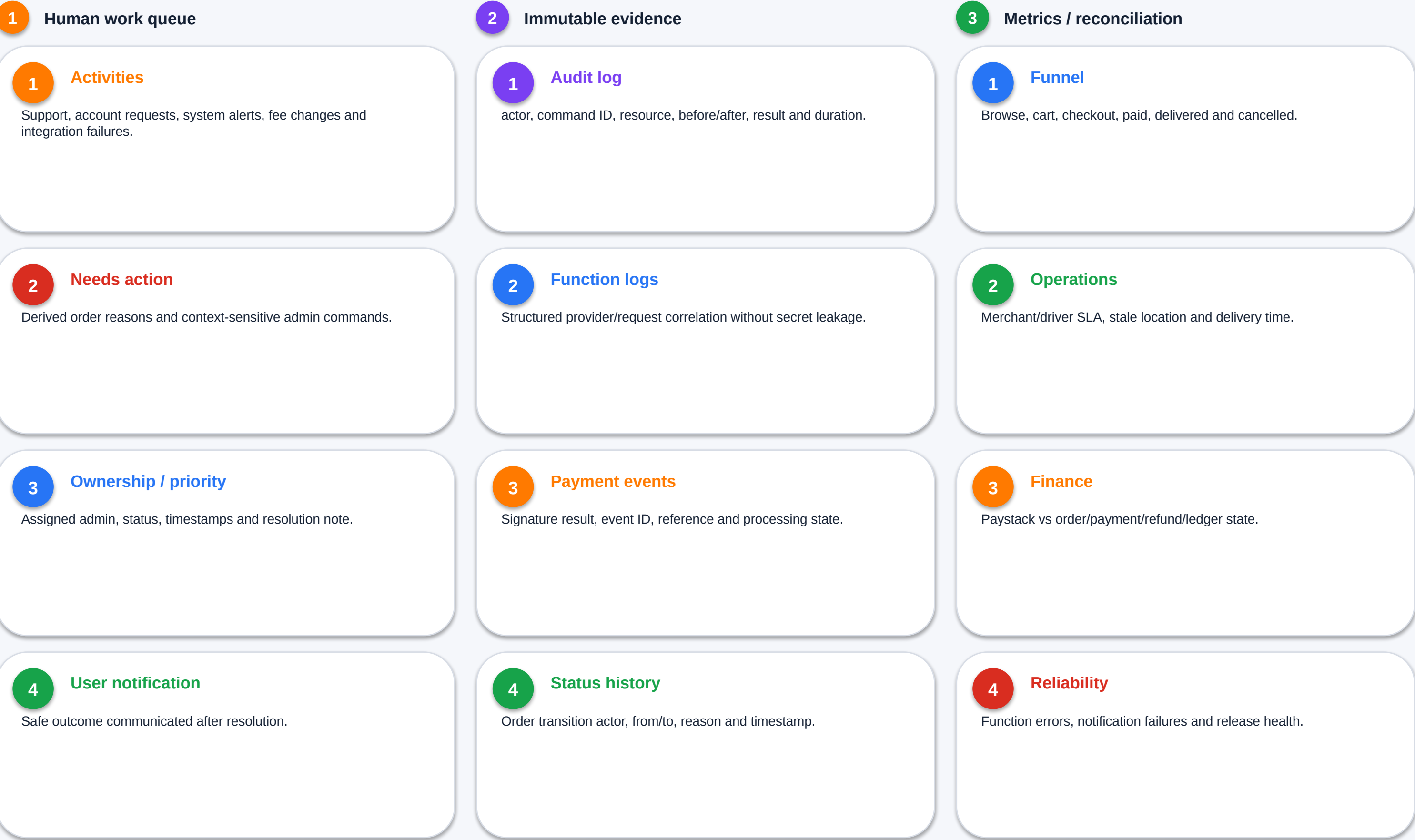
Defense in Depth and Sensitive Commands

Security is enforced from route to provider and produces immutable evidence.



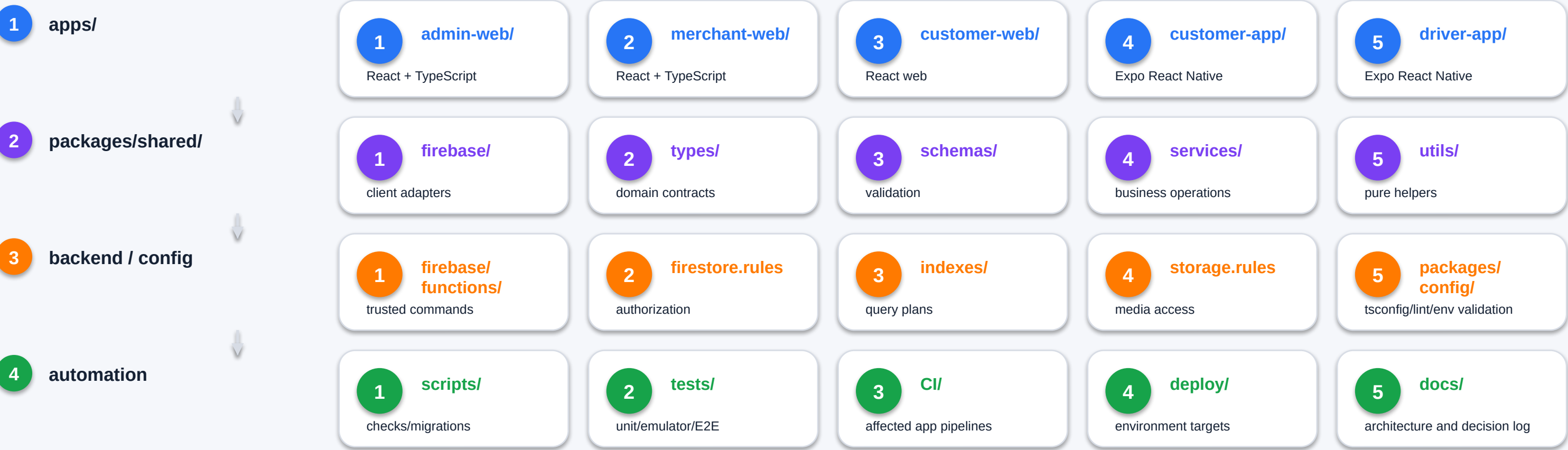
Activities, Audit Logs, Metrics and Reconciliation

Operational problems become visible work rather than silent client errors.



Five-App Workspace and Dependency Boundaries

The workspace consolidates code without creating app-to-app coupling.



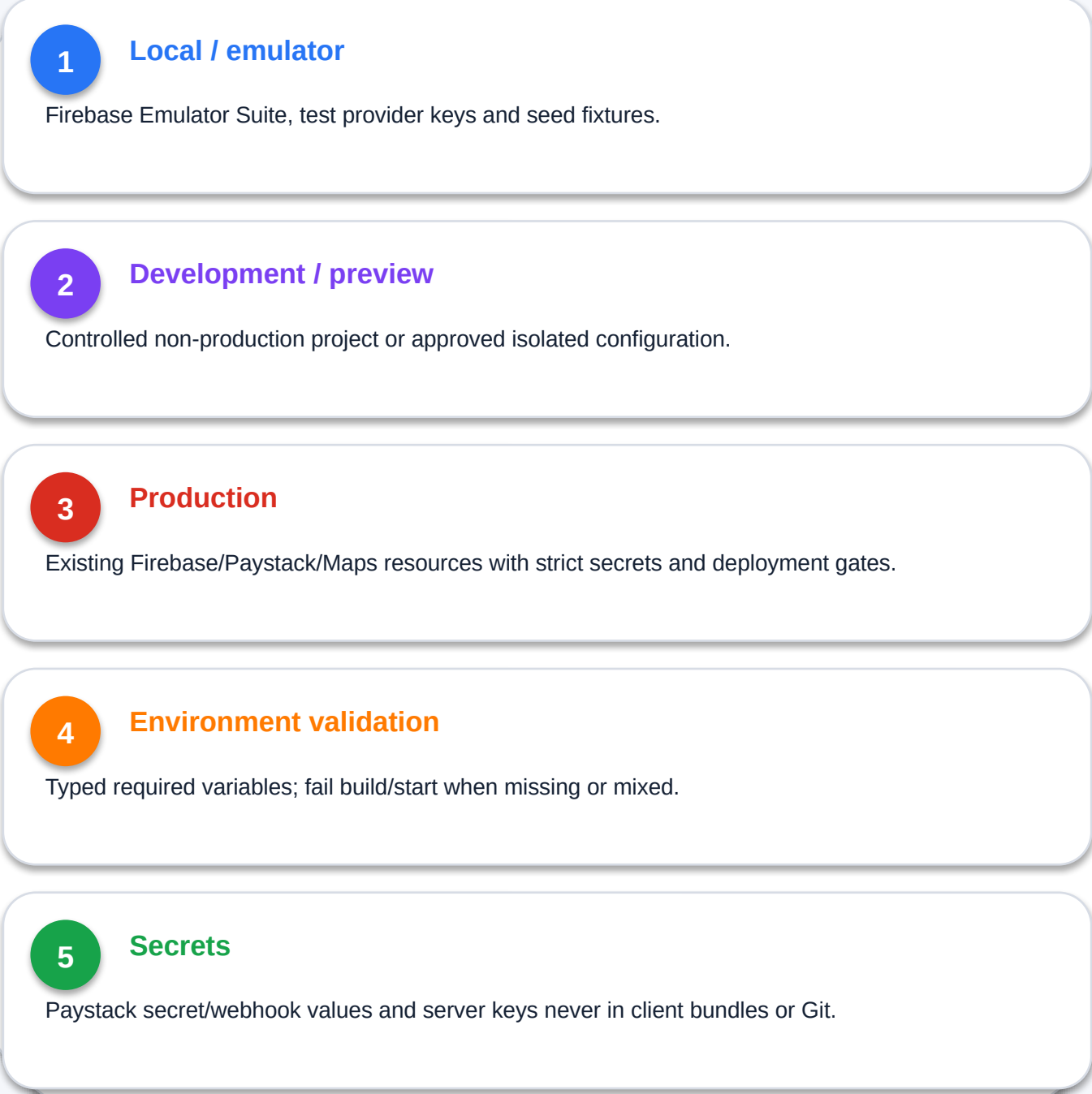
Dependency direction

apps/* -> packages/shared -> Firebase/external SDKs. No shared -> app imports, no app -> app imports and no forced web/mobile UI reuse.

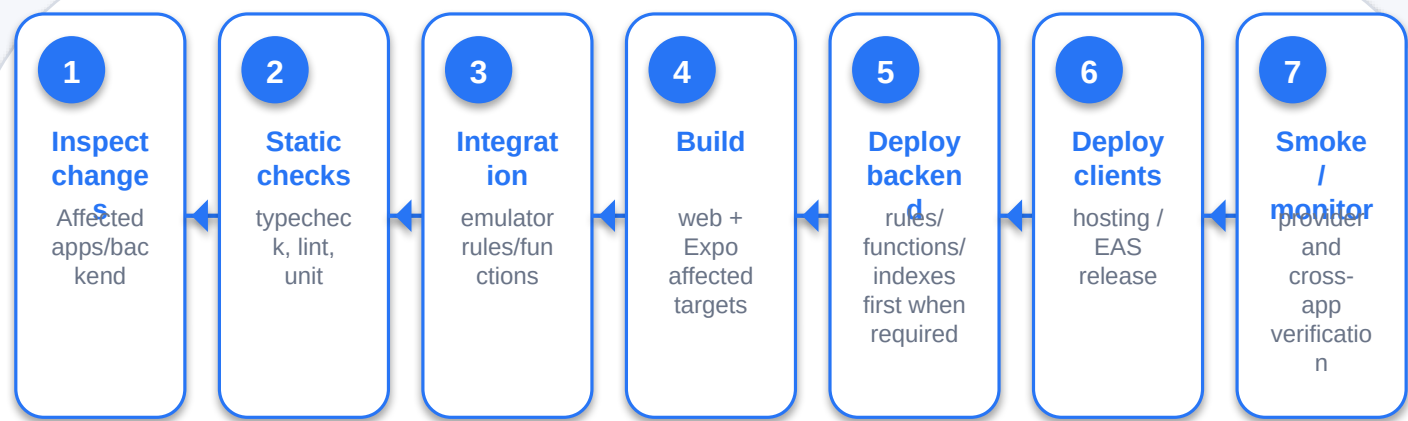
Configuration, Secrets, CI/CD and Deployment

Environment isolation must be explicit while preserving one coherent production platform.

1 Environment model



2 Delivery pipeline



- Deploy backend compatibility before clients that depend on new schema or commands.
- Use versioned migrations and backward-compatible readers during rollout.
- Record release version and schema/function deployment in the audit/change log.
- Rollback plans must consider provider/webhook and partially migrated data.

No accidental production mixing

Every app should display/log its environment identifier internally. CI validates Firebase project ID, Paystack mode and Maps configuration as a consistent set.

Listener, Query, Media, Maps and Cost Guardrails

Scale is achieved by bounded work and explicit freshness policies, not by making everything realtime.

1 Data and listeners

1 Scoped realtime

Active order/queue only; unsubscribe on route/tab exit.

2 Pagination

Server-backed histories, catalogs and admin lists.

3 Indexes

Design composite indexes for actual filters and date ranges.

4 Snapshots

Denormalized order display data avoids repeated joins.

2 Assets and maps

1 Images

Compress, thumbnail, lazy-load and safely clean orphans.

2 Maps

Lazy-load panels, cache routes within freshness window.

3 Driver location

Throttle by movement/time/state; stop when completed.

4 Provider calls

Backend caching/deduplication and quotas with observability.

3 Application bundles

1 Web splitting

Route-level chunks and no mobile/admin library leakage.

2 Mobile lists

Virtualized rendering and restrained animations/blur.

3 Cache policy

Catalog/cacheable data yes; payment truth no.

4 Cost signals

Reads, Functions, Storage, FCM, Maps and provider volumes measured.

Idempotency, Transactions, Retry and Cleanup

Every money-changing or lifecycle-changing operation is safe under duplication, interruption and partial failure.

1 Reliability patterns

1 Stable IDs
checkoutId, payment reference, provider event ID, command ID and importBatchId.

2 Transactions / batches
Canonical state, status history, audit and outbox move together.

3 Optimistic limits
Only reversible low-risk UI edits; never paid/refunded/delivered truth.

4 Retry policy
Exponential backoff for transient providers; typed final failure.

5 Schema compatibility
Backward-compatible readers and explicit migration scripts.

2 Automated safeguards

1 Payment reconciliation
Verify stuck processing and mismatched paid/refund states.

2 SLA monitors
Merchant response, assignment, delay and stale location activities.

3 Cleanup
Expired quotes, abandoned imports, invalid tokens and orphaned media.

4 Data consistency
Detect role/claim/scope disagreement and invalid references.

5 Backup/export policy
Document recovery objectives before destructive migrations.

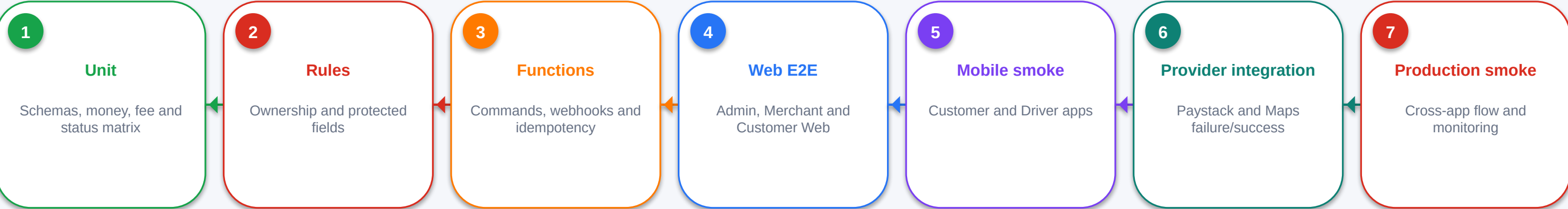
Fallback honesty

Maps/payment/notification failures must not silently fabricate successful state. Surface a retry, safe block or admin activity.

Cross-App Test Matrix and Production Acceptance

Testing follows the user journey and the security boundary, not only individual screens.

1 Test layers



2 Critical acceptance scenarios

Scenario	Apps / services	Pass condition
Guest -> paid order	Customer + Functions + Paystack	Exactly one paid order and merchant notification
Merchant fulfillment	Merchant + order + notifications	Allowed transitions visible to all channels
Driver assignment/delivery	Admin + Driver + Maps + Customer	Race-safe assignment and scoped tracking
Payment webhook replay	Paystack handler + Firestore	No duplicate order/value/status mutation
Cross-role attack	Rules + Functions	Unauthorized read/write denied and logged
Refund	Admin + Paystack + ledger	Amount/state validated and provider result reconciled
Account deletion	Customer + Admin + Auth + cleanup	Login disabled, PII redacted, history retained
Release compatibility	Old/new client + schema	No breaking partial-deploy failure

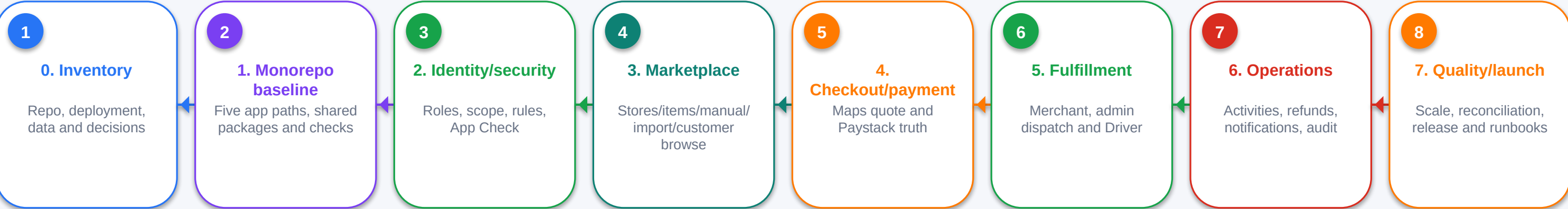
Production gate

No release passes on screenshots alone. Require static checks, emulator tests, affected builds, provider test evidence, manual role smoke and a documented rollback/monitoring plan.

Start-to-End Implementation Roadmap

This sequence reduces platform risk by stabilizing shared contracts and core transaction truth before advanced automation.

1 Major phases



2 Phase deliverables and gates

Phase	Primary deliverables	Exit gate
0. Inventory	File map, Firebase paths, Functions, rules, EAS IDs, current tests	No unresolved source-of-truth conflict
1. Monorepo	apps/* + packages/shared + scripts/config without breaking builds	All existing apps compile/run
2. Security	Role/status/scope contract, rule tests, command boundary, App Check plan	Cross-role denial verified
3. Marketplace	Admin + Merchant store/catalog, Customer browse, Storage media	Manual creation remains; active catalog consistent
4. Checkout	Address, Routes/Matrix, fee quote, Paystack initialize/verify/webhook	Exactly one paid order per reference
5. Fulfillment	Merchant queue, lifecycle, assignment, Driver pickup/location/delivery	End-to-end paid -> delivered works
6. Operations	Needs-action, support, refund, account lifecycle, audit/notifications	Failures visible and financially reconciled
7. Launch	Performance, backups, monitoring, CI/CD, runbooks, staged deployment	Production acceptance matrix passes

Scope discipline

Each phase is decomposed into bounded features. Do not combine monorepo migration, schema redesign, payment integration and UI restyling in one Codex task.

Required Prompt Contract for Every Development Task

Use this checklist at the beginning and end of each bounded Codex task.

1 Preflight

1 Inspect current state

Repo files, git status, deployed behavior and relevant blueprint pages.

2 Summarize completed work

Do not restart, revert or repeat accepted progress.

3 Declare scope

Exact app/module/files/data paths and feature boundary.

4 Impact analysis

Schema, rules, Functions, indexes, environments and deployment.

2 Change constraints

! No second backend

Preserve Firebase project, collections and EAS IDs.

! No rule loosening

Fix architecture or command boundary instead.

! No unrelated refactor

Patch bounded scope and preserve working methods.

! Safe sharing only

No app-to-app UI imports or premature abstraction.

3 Verification/report

1 Targeted tests

Unit/emulator/E2E for changed behavior.

2 Static/build

Typecheck, lint and affected app builds.

3 Firebase validation

Rules, indexes, Functions and provider configuration.

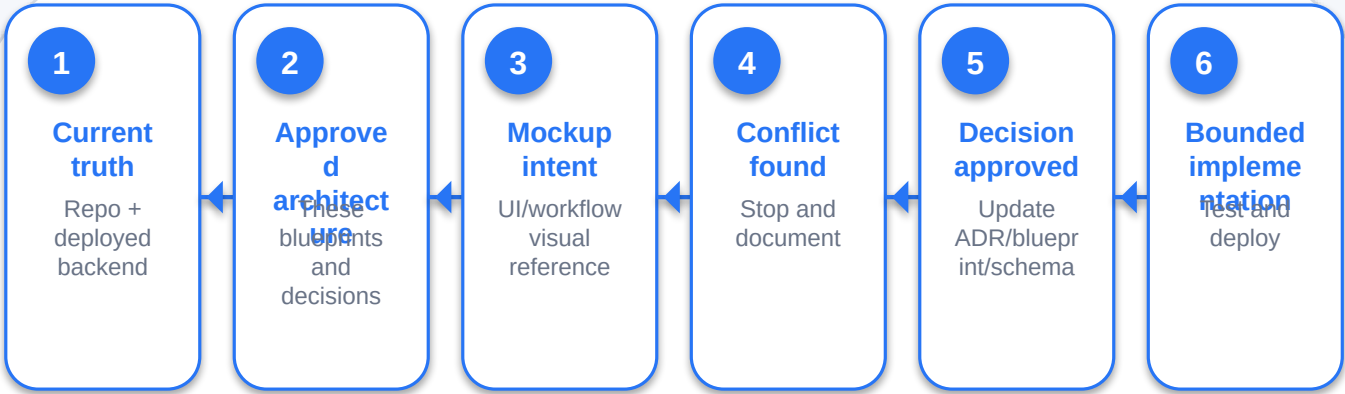
4 Complete report

Files, schema/deploy changes, manual tests, risks and next step.

Source Precedence, Assumptions and Change Governance

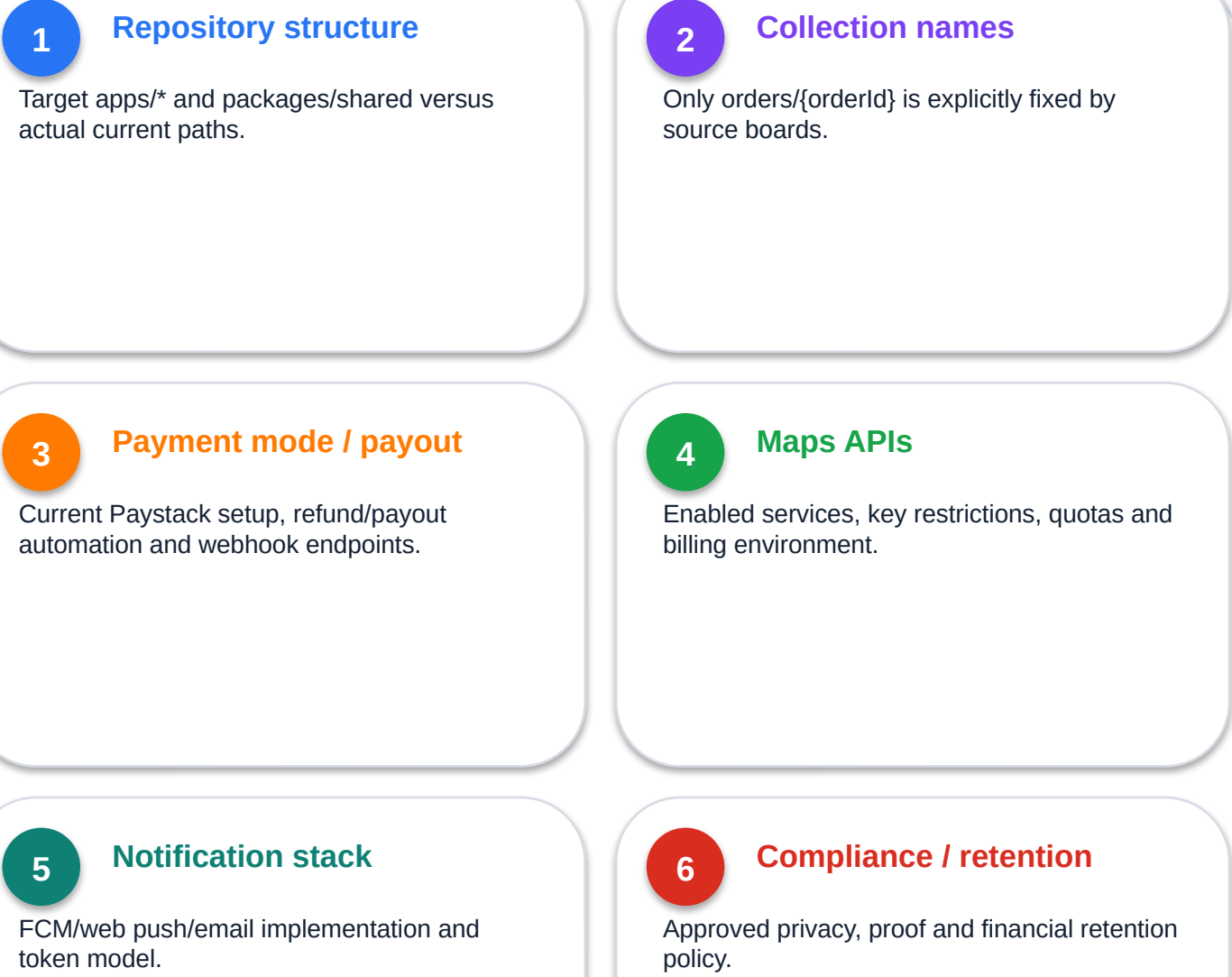
Architecture evolves through explicit decisions rather than silent divergence between apps.

1 Source precedence and change process



- Do not guess collection names, provider modes, current Functions or route paths.
- An architecture change names affected apps, data, security, migration, rollout and rollback.
- Deprecations use compatibility windows; clients are not broken by immediate schema replacement.
- The master plan is updated when a decision changes a cross-app contract.

2 Current assumptions to confirm



Architecture status

Main implementation plan from development start to production operation. Exact repo and cloud configuration remains the final technical truth and must be inspected before code changes.

Master Plan Traceability and Current Decisions

Traceability, current decisions and implementation assumptions.

1

Source-to-module traceability

Source	Captured architecture detail	Used in this document
Admin Web Architecture Blueprint	System context, UI, roles, stores/items, orders, activities, data, security and monorepo	Foundation for all pages
Current Projectweb deployments	Customer, Driver and Admin observed behavior and role experiences	Channel-specific flows
Monorepo migration plan	Five apps, shared package boundaries, one Firebase project and preserved EAS IDs	Pages 20-26
Store/item import decisions	Manual workflows remain first-class; imports require preview/review	Pages 17, 25
Account lifecycle decision	Suspend/unsuspend and archive/redact instead of destructive history deletion	Pages 2, 10, 17, 24
Official Paystack documentation	Backend initialization, secret protection, verification, amount checks and webhooks	Page 14
Firebase documentation	Auth, Firestore, Functions, Storage, Messaging, Hosting and App Check roles	Pages 9-12
Google Maps Routes documentation	Route, route matrix and normalized Maps enrichment	Page 13

2

Normalized decisions

Backend	One Firebase project and one canonical data model.
UI stacks	React web for web apps; Expo React Native for mobile.
Pricing V1	Mabopane, standard delivery, no surge.
Payment	Paystack server initialization and verified asynchronous truth.
Dispatch V1	Manual admin assignment; future automation fits same contract.
Deletion	Archive/redact + Auth disable; preserve required records.
Manual onboarding	Manual stores/items remain available beside import methods.
Source precedence	Repo/deployment > approved plan > mockups.

Document status

Architecture synthesis for implementation. Before changing code, validate exact collection names, existing Cloud Functions, deployed rules, current routes and environment variables against the monorepo.